

CPSC 440/550 Advanced Machine Learning (Jan-Apr 2025)
Assignment 3 – due Thursday April 9 at **11:59pm**

The assignment instructions are the same as for the previous assignment.

Mitchell Polley, poll8030, Student Number: 11983863

1 Bayesian Logistic Regression [50 points]

`main.py blogreg-mcmc` loads a set of images of ‘2’ and ‘3’ digits. It then runs the Metropolis MCMC algorithm to try to generate samples from the posterior over w , in a logistic regression model with a Gaussian prior. Once the samples are generated, it makes a histogram of the samples for several of the variables. The plots also show the MAP estimate, as a vertical line.¹

[1.1] [5 points] Why might the samples coming from the Metropolis algorithm as run here not give a good approximation to the posterior?

Answer: From running the script and inspecting the plot, the Metropolis algorithm is stuck at its initial value of 0 and every sample generated was rejected (yielding a 0% acceptance rate). This happens because of the curse of dimensionality and our proposal distribution’s large step size. The proposal adds standard normal noise ($\mathcal{N}(0, I)$) to the weights. Because the parameter space is high-dimensional, moving by a variance of 1 in every dimension simultaneously results in a big, undirected/random jump. In a high-dimensional space, these large jumps very likely land in regions of extremely low posterior probability, causing the acceptance ratio to drop to zero.

[1.2] [5 points] Modify the proposal used by the demo to $\hat{w} \sim \mathcal{N}(w, (1/10)^2 \mathbf{I})$ instead of $\hat{w} \sim \mathcal{N}(w, \mathbf{I})$. Hand in your modifications to the code and the updated histogram plot.

Answer:

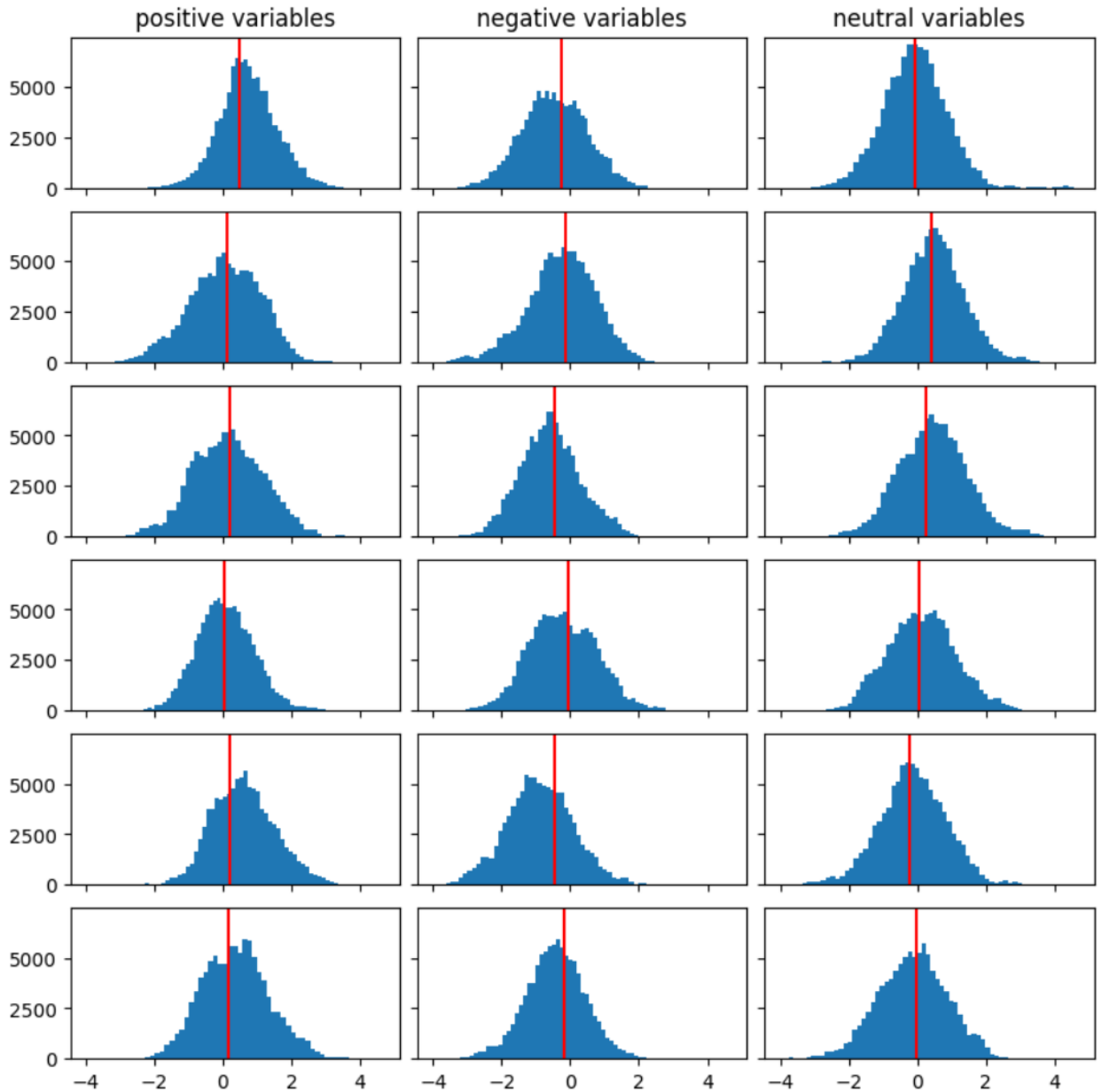
```
1 def sample_weights(self, X, y, n_samples, rng=None):
2     if rng is None:
3         rng = np.random.default_rng()
4
5     n, d = X.shape
6     log_prob = self._get_log_prob(X, y)
7
8     samples = np.full((n_samples, d), np.nan)
9     w = np.zeros(d)
10    log_p = log_prob(w)
11
12    scale_factor = 1e-1 # added scale factor to make acceptance rate reasonable
13
14    n_accept = 0
15    pbar = tqdm(range(n_samples))
16    for i in pbar:
17        w_hat = w + rng.normal(
18            loc=0,
19            scale=scale_factor,
20            size=d,
21        )
22        log_phat = log_prob(w_hat)
23
24        log_r = log_phat - log_p
25        if np.log(rng.uniform()) < log_r:
26            w = w_hat
27            n_accept += 1
28            log_p = log_phat
29        samples[i, :] = w
30        pbar.set_postfix({"accept_rate": f"{n_accept / (i + 1):.2%}"})
31
```

¹The division into “positive,” “negative,” and “neutral” variables is based on the sign of that MAP estimate.

```

32 print(
33     f"did {n_samples:,} steps; accepted {n_accept:,} samples ({n_accept /
34         ↪ n_samples:.2%})"
35 )
36 return samples

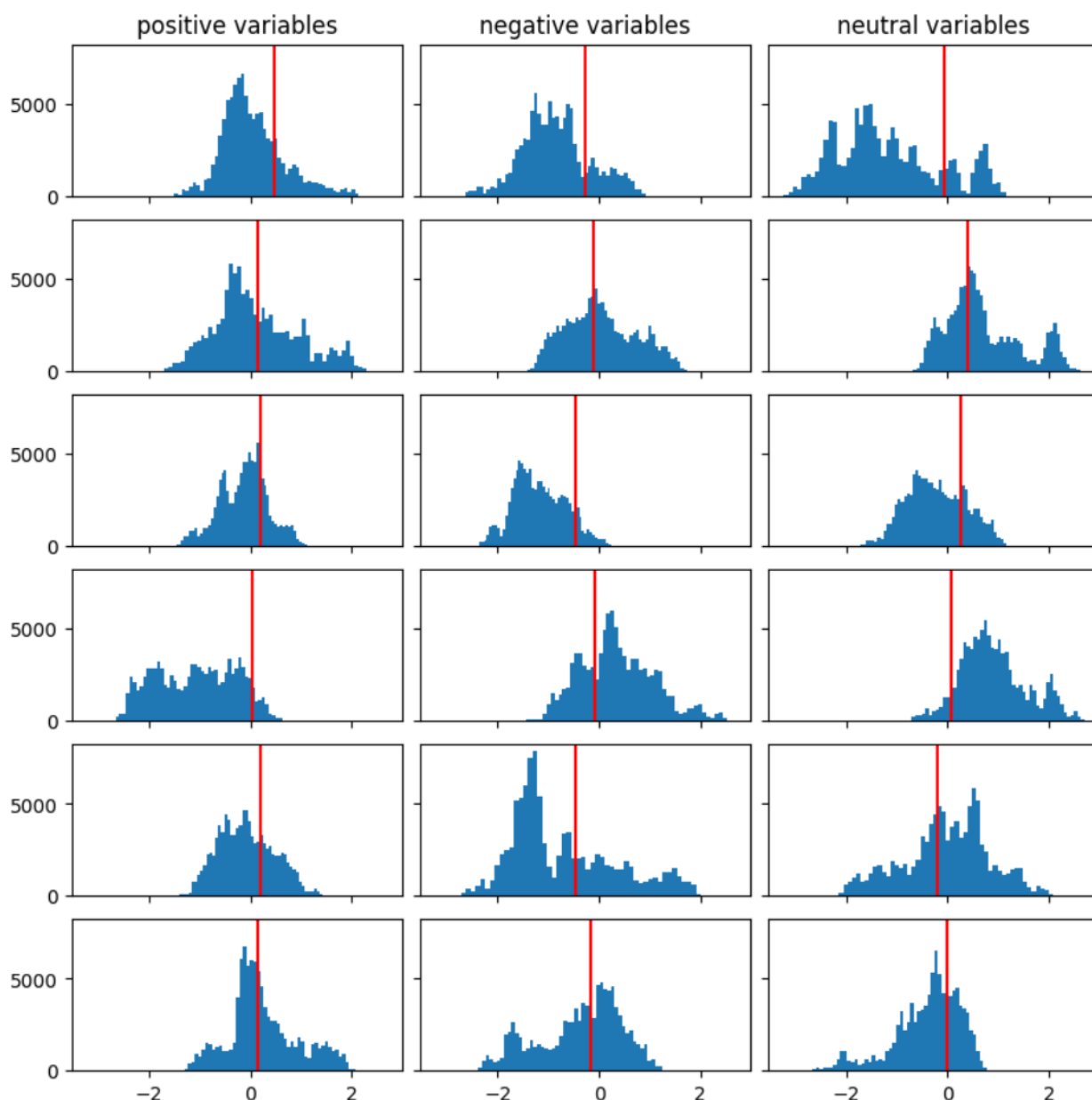
```



[1.3] [5 points] Modify the proposal to use $\hat{w} \sim \mathcal{N}(w, (1/100)^2 \mathbf{I})$. Hand in your plots; do you think this is probably sampling from the posterior distribution more or less effectively than the previous choice? (Briefly explain.) (Don't base your answer on the test likelihoods, those are for later!)

Answer: I believe the modification in 1.2 of $\sigma = 10^{-1}$ (acceptance rate = 30%) is a better choice and σ

$= 10^{-2}$ results in a higher acceptance rate of $\geq 90\%$ but worse distribution around the MAP estimate for each feature. You can see that the samples of w with $\sigma = 10^{-1}$ are, upon visual inspection, normally distributed. While those samples generated with a lower $\sigma = 10^{-2}$ are less normally distributed.



- [1.4] [5 points] Go back to $\hat{w} \sim \mathcal{N}(w, (1/10)^2 \mathbf{I})$ but change the code to initialize at the MAP solution, rather than at zero. Hand in the code changes; do you think this meaningfully improved sampling compared to initializing at zero? (Briefly explain.) (Again, don't base your answer on the test likelihoods, those are for later!)

Answer: Yes, initializing at the MAP estimate meaningfully improves the sampling. You can see by comparing the the old histograms to the new ones that the new distributions are smoother, less skewed, and centred around the MAP estimate. The MAP estimate is the peak/mode of the target

posterior distribution. When we initialize the weights at zero, the MCMC chain starts in a low-probability region and has to waste many early iterations slowly migrating toward higher-probability areas or "burn-in". These early samples don't represent the true distribution. By initializing directly at the MAP estimate, placing our starting point exactly at the highest probability region, the chain immediately begins exploring the most relevant areas of the posterior without needing a burn-in period. This gives a much more representative samples of the true samples of the posterior right from the first step.

```

1  def sample_weights(self, X, y, n_samples, map_estimate=None, rng=None):
2      if rng is None:
3          rng = np.random.default_rng()
4
5      n, d = X.shape
6      log_prob = self._get_log_prob(X, y)
7
8      samples = np.full((n_samples, d), np.nan)
9      w = np.zeros(d) if map_estimate is None else map_estimate
10     log_p = log_prob(w)
11
12     scale_factor = 1e-1 # added scale factor to make acceptance rate reasonable
13
14     n_accept = 0
15     pbar = tqdm(range(n_samples))
16     for i in pbar:
17         w_hat = w + rng.normal(
18             loc=0,
19             scale=scale_factor,
20             size=d,
21         )
22         log_phat = log_prob(w_hat)
23
24         log_r = log_phat - log_p
25         if np.log(rng.uniform()) < log_r:
26             w = w_hat
27             n_accept += 1
28             log_p = log_phat
29             samples[i, :] = w
30             pbar.set_postfix({"accept_rate": f"{n_accept / (i + 1):.2%}"})
31
32     print(
33         f"did {n_samples:,} steps; accepted {n_accept:,} samples ({n_accept /
34         ↪ n_samples:.2%})"
35     )
36     return samples
37
38 @handle("blogreg-all")
39 def blogreg(mcmc=True, var=True):
40     X, y = load_dataset("two_threes", "X", "y")
41
42     rng = np.random.default_rng(seed=12)
43     is_train = rng.random(size=X.shape[0]) < 0.8
44     X_train = X[is_train]
45     y_train = y[is_train]
46     X_test = X[~is_train]
47     y_test = y[~is_train]

```

```

47 model = BayesianLogisticRegression()
48 map_est = model.map_estimate(X_train, y_train)
49
50
51 if mcmc:
52     mcmc_samples = model.sample_weights(X_train, y_train,
53     ↪ n_samples=100_000, map_estimate=map_est)
54     model.plot_weights(mcmc_samples, map_est, figname="blogreg-mcmc")
55
56 if var:
57     var_mn, var_L = model.variational_approx(X_train, y_train)
58     var_samples = var_mn + np.random.randn(100_000, var_mn.shape[0]) @ var_L
59     model.plot_weights(var_samples, map_est, figname="blogreg-var")
60
61 map_test = model.different_log_likelihoods(X_test, y_test, map_est[np.newaxis])
62 print(f"Test log-likelihood: MAP {map_test}")
63
64 if mcmc:
65     mcmc_test = model.different_log_likelihoods(X_test, y_test, mcmc_samples)
66     print(f"                MCMC {mcmc_test}")
67
68 if var:
69     var_test = model.different_log_likelihoods(X_test, y_test, var_samples)
70     print(f"                var {var_test}")

```

- [1.5] [15 points] The method `BayesianLogisticRegression.variational_approx` has most of an implementation of a variational approximation to the posterior distribution over weights as a multivariate Gaussian. Finish the implementation; [hand in your code and the histogram plots](#).

Answer:

```

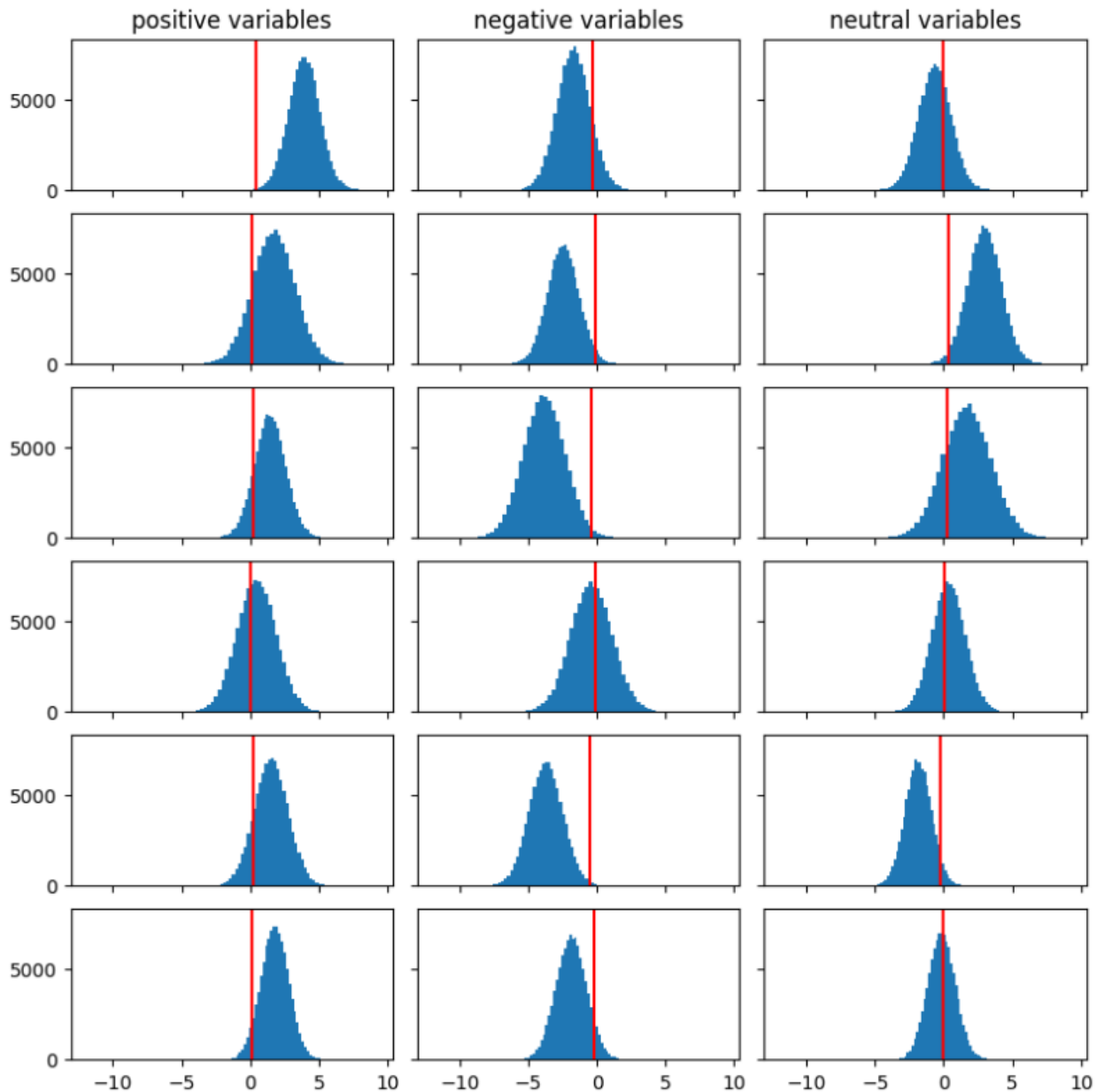
1 def variational_approx(self, X, y, batch_size=128, train_steps=5_000):
2     log_prob = self._get_log_prob(X, y)
3
4     d = X.shape[1]
5     tril_inds = tuple(torch.tril_indices(d, d, offset=-1))
6     diag_inds = (range(d), range(d))
7
8     mu = torch.zeros(d, requires_grad=True)
9     L_log_diag = torch.zeros(d, requires_grad=True)
10    L_tril = torch.zeros(tril_inds[0].shape[0], requires_grad=True)
11
12    def get_L():
13        L = torch.zeros(d, d)
14        L[diag_inds] = L_log_diag.exp()
15        L[tril_inds] = L_tril
16        return L
17
18    def variational_obj():
19        # should return the objective to *maximize*
20        # (using a Monte Carlo estimate with batch_size samples)
21        L = get_L()
22
23        #sample eps ~ N(0, I) for batch size and reparameterize to get samples of w
24        eps = torch.randn(batch_size, d)
25        w_samples = mu + eps @ L.T # [batch_size, d]
26

```

```

27     log_probs = log_prob(w_samples.T) # [batch_size]
28
29     log_joint = log_probs.mean() # monte carlo estimate of  $E_q[\log p(w, D)]$ 
30     log_entropy = L_log_diag.sum() + 0.5 * d * (1 + np.log(2 * np.pi)) # entropy of
    ↪  $q(w)$  + some minor constant to avoid floating point issues
31     return log_joint + log_entropy
32
33     # LBFGS is too slow here with the higher-dimensional problem
34     opt = torch.optim.Adam([mu, L_log_diag, L_tril], lr=0.1)
35     pbar = tqdm(range(train_steps))
36
37     obj_avg = variational_obj().detach().item()
38     for it in pbar:
39         opt.zero_grad()
40
41         obj = variational_obj()
42         (-obj).backward() # minus sign to minimize instead
43
44         obj_avg = 0.9 * obj_avg + 0.1 * obj.detach().item()
45         pbar.set_postfix({"obj_avg": obj_avg})
46
47         opt.step()
48
49     return (p.detach().numpy() for p in [mu, get_L()])

```



[1.6] [15 points] The `BayesianLogisticRegression.different_log_likelihoods` method implements four different attempts at evaluating the predictive log-likelihood of this method on test data. Each one has a different interpretation. Explain each computation; don't just directly translate the code into English, but say what it's computing in words. Which is the correct log-likelihood for the fully-Bayesian prediction?

Answer:

Method 1: Smoothed majority vote. This method treats the posterior ensemble as a committee of classifiers. For each test point, every posterior sample makes a hard binary prediction (checking if $x \cdot w > 0$), and then we calculate the fraction of samples that voted for the correct class. A small Laplace smoothing factor (+0.1 and +0.2) is added to prevent any fraction from being exactly 0 or 1,

which would break the log calculation. Finally, we take the log of these smoothed fractions and sum them across all test points. The flaw here is that it completely discards the probability of the model by forcing each sample into a hard binary vote before aggregating.

Method 2: Average of per-point log-likelihoods. For each test point, we compute the log-probability of the correct label under each sample, average those log-probabilities across all samples, and then sum across all test points. Mathematically, this computes $\sum_i \frac{1}{S} \sum_s \log p(y_i|x_i, w_s)$. Because we are averaging in log space, we are computing $\mathbb{E}_q[\log p(y_i|x_i, w)]$ for each point. By Jensen’s inequality, since the log function is strictly concave ($\mathbb{E}[\log p] \leq \log \mathbb{E}[p]$), this underestimates the true predictive log-likelihood.

Method 3: Average of per-sample total log-likelihoods. Instead of looking at individual points first, this evaluates how well a single sampled weight vector w explains the entire test dataset. We compute the total log-likelihood across all test points for a single sample, and then average that total across all S samples. Mathematically, this is $\frac{1}{S} \sum_s \sum_i \log p(y_i|x_i, w_s)$. Because summation and averaging are linear operations and commute, this method is numerically identical to Method 2. It asks a different conceptual question (“what is the expected total log-likelihood under the posterior?”) but yields the exact same number, meaning it suffers from the same Jensen’s inequality underestimation.

Method 4: Log of averaged predictive probabilities. This method computes the probability $p(y = 1|x_i, w_s)$ under each sample, but importantly, it averages these values in probability space rather than log space. This gives us the approximated Bayesian predictive probability: $p(y_i|x_i, \text{data}) \approx \frac{1}{S} \sum_s p(y_i|x_i, w_s)$. We then evaluate the final log-likelihood using these averaged probabilities: $\sum_i \log \left[\frac{1}{S} \sum_s p(y_i|x_i, w_s) \right]$.

$\therefore$ Method 4 is the correct fully-Bayesian predictive log-likelihood. The true Bayesian predictive distribution requires integrating out the weights across the posterior:

$$p(y^*|x^*, \mathcal{D}) = \int p(y^*|x^*, w)p(w|\mathcal{D})dw$$

Method 4 correctly approximates this integral by averaging the predictive probabilities across the posterior samples before applying the log. Methods 2 and 3 incorrectly average inside the log, which gives an underestimation of the model’s true predictive performance. Method 1 is the crudest, as it abandons probabilities entirely for hard votes.

2 Discrete Markov Chain Inference [50 points]

The function `grad_chain()` in `main.py` makes a Markov chain model corresponding to the “CS graduate Markov chain” from class.

Note that, slightly contrary to class, this question will use the notation that the first step is index 0; this will help avoid confusion between the math and zero-indexed Python code:

$$p(x_0, x_1, \dots, x_{d-1}) = p(x_0) \prod_{j=1}^{d-1} p(x_j \mid x_{j-1}).$$

- [2.1] [9 points] Implement the function `MarkovChain.sample`, in `markov_chain.py`, that uses ancestral sampling to sample sequences of a given length. [Hand in this code](#), and report the univariate marginal probabilities for time 30 using a Monte Carlo estimate based on 10,000 samples (from `main.py mc-sample`).

Hint: `rng.choice` might be helpful.

Answer:

∴ Empirical dist, time 30: [0.019 0.513 0.024 0.005 0.11 0.074 0.254]

```

1 def sample(self, n_samples, length, init_probs=None, rng=None):
2     # note that:
3     # length 1 should return only timestep 0
4     # length 2 should return timesteps 0, 1
5     # etc
6     if rng is None:
7         rng = np.random.default_rng()
8     if init_probs is None:
9         init_probs = self.init_probs
10
11     samples = np.zeros((n_samples, length), dtype=int)
12
13     #sample from initial distribution for each of the 10k chain init state
14     samples[:, 0] = rng.choice(self.num_states, size=n_samples, p=init_probs)
15
16     #sample from transition distribution
17     for j in range(n_samples):
18         for t in range(1, length):
19             samples[j, t] =
20                 ↪ rng.choice(self.num_states, p=self.transition_probs[samples[j, t-1]])
21
22     return samples

```

- [2.2] [9 points] Write a function, `MarkovChain.marginals`, that uses the CK equations to compute the exact univariate marginals up to a given time t . Use `main.py mc-marginals` to find the exact marginals at time index 30. [Hand in this code](#), report the exact univariate marginals at time 30 (to three decimal places). (These should be close to the marginals from the previous part!)

Answer:

∴ Marginals at time 30: [0.019 0.514 0.023 0.006 0.107 0.072 0.26]

```

1 def marginals(self, length, init_probs=None):
2     if init_probs is None:
3         init_probs = self.init_probs

```

```

4         margs = np.zeros((self.num_states, length))
5
6
7         for t in range(length):
8             if t == 0:
9                 margs[:, t] = init_probs
10            else:
11                margs[:, t] = init_probs @ np.linalg.matrix_power(self.transition_probs, t)
12
13        return margs
14
15    @handle("mc-marginals")
16    def mc_marginals():
17        model = grad_chain()
18
19        time = 30
20        marginals = model.marginals(length=time + 1)
21        print("Marginals at time 30:", marginals[:, time].round(3))
22

```

[2.3] [2 points] What is the state c with highest marginal probability, $p(x_j = c)$, for each time j ?

Answer:

Time	State	Probability
0	INDUSTRY	0.600
1	INDUSTRY	0.678
2	INDUSTRY	0.698
3	INDUSTRY	0.708
4	INDUSTRY	0.712
5	INDUSTRY	0.713
6	INDUSTRY	0.711
7	INDUSTRY	0.706
8	INDUSTRY	0.700
9	INDUSTRY	0.693
10	INDUSTRY	0.686
11	INDUSTRY	0.677
12	INDUSTRY	0.669
13	INDUSTRY	0.660
14	INDUSTRY	0.651
15	INDUSTRY	0.642
16	INDUSTRY	0.633
17	INDUSTRY	0.623
18	INDUSTRY	0.614
19	INDUSTRY	0.605
20	INDUSTRY	0.597
21	INDUSTRY	0.588
22	INDUSTRY	0.579
23	INDUSTRY	0.570
24	INDUSTRY	0.562
25	INDUSTRY	0.554
26	INDUSTRY	0.545
27	INDUSTRY	0.537
28	INDUSTRY	0.529
29	INDUSTRY	0.521
30	INDUSTRY	0.514

- [2.4] [10 points] Write a function, `MarkovChain.mode`, that uses the Viterbi decoding algorithm for Markov chains to find the optimal decoding for a length- T sequence. [Hand in this code and report the optimal decoding of the Markov chain for 50 years and for 100 years.](#)

Answer:

```
1 def mode(self, length, init_probs=None):
2     # again, length should include timestep 0
3     if init_probs is None:
4         init_probs = self.init_probs
5
6     # define a matrix of zeros of shape [self.num_states, length] containing the probability
7     #   of each state across time, and a matrix of zeros of shape [self.num_states, length]
8     #   containing the mode of each state across time
9     modes = np.zeros((self.num_states, length), dtype=int)
10    # find the marginals up to the given length
11    margs = self.marginals(length, init_probs) # to avoid unnecessarily recomputing marginals
12    #   at each time step.
```

```

12  # Bottoms up DP approach:
13
14  # DP storage (structure only; no recursion/fill yet)
15  best_path_prob = np.full((self.num_states, length), -np.inf, dtype=float)
16  backptr = np.full((self.num_states, length), -1, dtype=int)
17
18  # base case at t=0
19  best_path_prob[:, 0] = np.log(init_probs + 1e-300) # this is the joint probability of
20  ↪ being in each state at time 0 (which is just the initial distribution)
21  backptr[:, 0] = np.arange(self.num_states)
22  modes[:, 0] = np.arange(self.num_states)
23
24  # Need a matrix for
25  for t in range(1, length):
26      # calculate the joint probability of each state at time t
27      for s in range(self.num_states):
28          # calculate the probability of being in state s at time t given each possible
29          ↪ previous state
30          for prev_s in range(self.num_states):
31              prob = best_path_prob[prev_s, t-1] + np.log(self.transition_probs[prev_s, s]
32              ↪ + 1e-300)
33              if prob > best_path_prob[s, t]: #check if greater than the default -inf value
34              ↪ or the current best path probability for state s at time t
35                  best_path_prob[s, t] = prob
36                  backptr[s, t] = prev_s
37                  modes[s, t] = backptr[s, t]
38
39  # return the best path and it's respective log probability
40  best_final_state = np.argmax(best_path_prob[:, -1])
41
42  best_path = np.zeros(length, dtype=int)
43  best_path[-1] = best_final_state
44
45  for t in range(length-1, 0, -1):
46      best_path[t-1] = modes[best_path[t], t]
47
48  path_prob = best_path_prob[best_final_state, -1]
49
50  return best_path, np.exp(path_prob) # return the best path and the probability of that
51  ↪ path (convert from log prob to prob)

```

Mode Sequences at Selected Time Points

Time $t = 50$

Mode Sequence:

```

[INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY,
INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY,
INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY, INDUSTRY,

```



```

9         if past_state is None:
10             # conditioning on nothing
11             return self.marginals(target_idx + 1)[: , -1]
12         else:
13             past_idx, past_val = past_state
14             # want  $p(x_{\text{target\_idx}} = ? \mid x_{\text{past\_idx}} = \text{past\_val})$ 
15             # initialize a vector of zeros of shape [self.num_states] to hold the
16             ↪ probabilities and set conditioned state probability to 1
17             cond_state_prob = np.zeros(self.num_states)
18             cond_state_prob[past_val] = 1.0
19
20             return cond_state_prob @ np.linalg.matrix_power(self.transition_probs,
21             ↪ target_idx - past_idx)
22
23     else:
24         future_idx, future_val = future_state
25
26         raise NotImplementedError()

```

- [2.6] [5 points] Fix the following portion of `main.mc_conditionals_past` to estimate the same numbers with rejection sampling; use “plain” samples from the model, initialized according to the chain initialization probabilities. Also fix the computation of `acceptance_rate`. [Hand in your code and the output.](#)

Answer:

Probabilities at time 20, given time 5 is 2 (GRAD_SCHOOL):

[0.015 0.378 0.029 0.013 0.283 0.143 0.14]

MC estimates at time 20, given time 5 is 2 (GRAD_SCHOOL):

[0.015 0.373 0.028 0.012 0.284 0.148 0.141]

Acceptance rate: 9.26%

```

1 @handle("mc-conditionals-past")
2 def mc_conditionals_past():
3     model = grad_chain()
4
5     probs = model.conditional_prob(t := 20, past_state=(past_t := 5, past_v := 2))
6     print(
7         f"Probabilities at time {t}, given time {past_t} is {past_v}
8         ↪ ({model.state_names[past_v]}):",
9         probs.round(3),
10    )
11
12    num_samples = 100_000
13
14    #Generate full sequences up to the target time
15    samples = model.sample(num_samples, t + 1)
16
17    # find which samples match our past condition (x_5 = 2)
18    accepted_mask = samples[:, past_t] == past_v

```

```

19  #only keep accepted samples
20  accepted_samples = samples[accepted_mask]
21
22  # Calculate the acceptance rate
23  num_accepted = len(accepted_samples)
24  acceptance_rate = num_accepted / num_samples
25
26  # Calculate the empirical probabilities at time t from accepted samples
27  if num_accepted > 0:
28      # np.bincount counts occurrences of each state (0 to num_states-1)
29      counts = np.bincount(accepted_samples[:, t], minlength=model.num_states)
30      mc_probs = counts / num_accepted
31  else:
32      mc_probs = np.full(model.num_states, np.nan)
33
34  print(
35      f"MC estimates at time {t}, given time {past_t} is {past_v}
36      ↪ ({model.state_names[past_v]}):",
37      mc_probs.round(3),
38  )
39  print(f"\tacceptance rate: {acceptance_rate:.2%}")

```

- [2.7] [10 points] `main.py mc-conditionals-both` calls `model.conditional_prob` to find $p(x_{20} \mid x_5 = 2, x_{30} = 5)$. Finish the implementation of the rest of the `conditional_prob` method to give an exact answer with dynamic programming, so that this computation is correct; also finish the rest of `main.mc_conditionals_both` to estimate the same probabilities with rejection sampling (and correctly compute the acceptance rate). [Hand in your code for both parts, and the output.](#)

Answer:

$$p(x_{20} = i \mid x_5 = 2, x_{30} = 5)$$

$$p(x_{20} = i \mid x_5 = 2, x_{30} = 5) = \frac{[T^{15}]_{2,i} \cdot [T^{10}]_{i,5}}{\sum_j [T^{15}]_{2,j} \cdot [T^{10}]_{j,5}}$$

Probabilities at time 20, given time 5 is 2 (GRAD_SCHOOL) and time 30 is 5 (ACADEMIA):

[0.001 0.015 0.021 0.018 0.285 0.66 0.0]

MC estimates at time 20, given time 5 is 2 (GRAD_SCHOOL) and time 30 is 5 (ACADEMIA):

[0.001 0.012 0.023 0.017 0.277 0.67 0.0]

Acceptance rate: 1.57%

```

1  def conditional_prob(self, target_idx, future_state=None, past_state=None):
2      # target_idx: the time index of the state we care about
3      # future_state: None or (future_idx, future_val)
4      # past_state: None or (past_idx, past_val)
5      #
6      # should return an array of shape [self.num_states] giving probabilities for each
7      ↪ state:

```



```

7      # return_val[i] = p(x_{target_idx} = i | x_{future_idx} = future_val, x_{past_idx}
      ↪ = past_val)
8      if future_state is None:
9          if past_state is None:
10             # conditioning on nothing
11             return self.marginals(target_idx + 1)[: , -1]
12         else:
13             past_idx, past_val = past_state
14             # want p(x_{target_idx} = ? | x_{past_idx} = past_val)
15             # initialize a vector of zeros of shape [self.num_states] to hold the
            ↪ probabilities and set conditioned state probability to 1
16             cond_state_prob = np.zeros(self.num_states)
17             cond_state_prob[past_val] = 1.0
18
19             return cond_state_prob @ np.linalg.matrix_power(self.transition_probs,
            ↪ target_idx - past_idx)
20
21     else:
22         future_idx, future_val = future_state
23
24         # Likelihood: p(x_{future_idx} = future_val | x_{target_idx} = i) = [T^{10}]_{i,
            ↪ 0}
25         steps_to_future = future_idx - target_idx
26         T_to_future = np.linalg.matrix_power(self.transition_probs, steps_to_future)
27
28         # extract the column corresponding to the known future value
29         likelihoods = T_to_future[:, future_val]
30
31         if past_state is None:
32             # Prior: p(x_{target_idx} = i) with no past knowledge
33             prior_probs = self.marginals(target_idx + 1)[: , -1]
34         else:
35             past_idx, past_val = past_state
36             # p(x_{target_idx} = i | x_{past_idx} = past_val) = [T^{15}]_{2, i}
37             steps_from_past = target_idx - past_idx
38             T_from_past = np.linalg.matrix_power(self.transition_probs, steps_from_past)
39
40             # Extract the row corresponding to the known past value
41             prior_probs = T_from_past[past_val, :]
42
43
44         numerator = prior_probs * likelihoods # this is the marginal of the target state
            ↪ conditioned on the past state (if given)
45
46         # Denominator = sum of numerator (equivalent to sum_j [T^{15}]_{2, j} *
            ↪ [T^{10}]_{j, 0})
47         denominator = np.sum(numerator)
48
49         return numerator / denominator
50
51
52 @handle("mc-conditionals-both")
53 def mc_conditionals_both():
54     model = grad_chain()
55
56     probs = model.conditional_prob(

```

```

57     t := 20,
58     past_state=(past_t := 5, past_v := 2),
59     future_state=(future_t := 30, future_v := 5),
60 )
61 print(
62     f"Probabilities at time {t}, given time {past_t} is {past_v}
63     ↪ ({model.state_names[past_v]}) ",
64     f"and time {future_t} is {future_v} ({model.state_names[future_v]}):\n",
65     probs.round(3),
66 )
67
68 # REJECTION SAMPLING
69 num_samples = 100_000
70
71 # Generate full sequences up to the future time
72 samples = model.sample(num_samples, future_t + 1)
73
74 # find which samples match our past condition AND future condition
75 accepted_mask = (samples[:, past_t] == past_v) & (samples[:, future_t] == future_v)
76
77 # only keep accepted samples
78 accepted_samples = samples[accepted_mask]
79
80 # Calculate the acceptance rate
81 num_accepted = len(accepted_samples)
82 acceptance_rate = num_accepted / num_samples
83
84 # Calculate the empirical probabilities at time t from accepted samples
85 if num_accepted > 0:
86     # np.bincount counts occurrences of each state (0 to num_states-1)
87     counts = np.bincount(accepted_samples[:, t], minlength=model.num_states)
88     mc_probs = counts / num_accepted
89 else:
90     mc_probs = np.full(model.num_states, np.nan)
91
92 print(
93     f"MC estimates at time {t}, given time {past_t} is {past_v}
94     ↪ ({model.state_names[past_v]}) ",
95     f"and time {future_t} is {future_v} ({model.state_names[future_v]}):\n",
96     mc_probs.round(3),
97 )
98 print(f"\tacceptance rate: {acceptance_rate:.2%}")

```